```
.(venv) (base) Sergio@SERHIIs-MacBook-Pro automation % python -m pytest
-v 2>&1 | tee evidence/pytest_output.txt
============================= test session starts
==============================
platform darwin -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0
rootdir: /Users/Sergio/Documents/QA_Embedded/Gate/Task6_a/
webinar_6_python_automation/homework_buggy/automation
configfile: pytest.ini
testpaths: tests
plugins: metadata-3.1.1, html-4.2.0
collected 96 items

tests/test_boundaries.py ....F...                                    [  8%]
tests/test_candidates_bugs.py F.F..FFF.....                          [ 26%]
tests/test_exploratory.py F.............ss...                        [ 46%]
tests/test_extra.py .....................FFFF.FFF......xxxx.....      [ 93%]
tests/test_relay.py .F                                               [ 95%]
tests/test_sensor.py F                                               [ 96%]
tests/test_status.py F.                                              [ 98%]
tests/test_timeout.py E                                              [100%]

==================================== ERRORS
====================================
_____________ ERROR at setup of test_no_response_becomes_timeout
_____________

request = <SubRequest 'device' for <Function
test_no_response_becomes_timeout>>

    @pytest.fixture
    def device(request):
        port = os.getenv("DEVICE_PORT")
        if not port:
            pytest.skip("Set DEVICE_PORT, e.g. COM5, /dev/ttyUSB0 or rfc2217://
localhost:4000 (Wokwi)")
        dev = Device(port, timeout=1.0)
        try:
            # Controlled precondition: finish boot, then known safe state.
            dev.wait_ready(READY_WAIT)
            try:
>               force_safe_state(dev)

conftest.py:59:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
conftest.py:44: in force_safe_state
    dev.command("RESET")
device.py:86: in command
```

```
    raw = self.command_raw(command, timeout=timeout)
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <device.Device object at 0x7fdd74239880>, command = 'RESET'
timeout = None

    def command_raw(self, command, timeout=None):
        """Send one command and return the first reply line.
        `timeout` overrides the read timeout for this call only."""
        old_timeout = self.ser.timeout
        if timeout is not None:
            self.ser.timeout = timeout
        try:
            self.ser.reset_input_buffer()
            self.log("TX", command)
            start = time.monotonic()
            self.ser.write((command + "\n").encode())
            self.ser.flush()
            line = self.ser.readline().decode(errors="replace").strip()
            elapsed = time.monotonic() - start
        finally:
            self.ser.timeout = old_timeout
        if not line:
            self.timed_out = True
            self.log("RX", f"<no response, timeout after {elapsed:.2f}s>")
>           raise TimeoutError(f"No response for command: {command}")
E           TimeoutError: No response for command: RESET

device.py:80: TimeoutError

During handling of the above exception, another exception occurred:

request = <SubRequest 'device' for <Function
test_no_response_becomes_timeout>>

    @pytest.fixture
    def device(request):
        port = os.getenv("DEVICE_PORT")
        if not port:
            pytest.skip("Set DEVICE_PORT, e.g. COM5, /dev/ttyUSB0 or rfc2217://
localhost:4000 (Wokwi)")
        dev = Device(port, timeout=1.0)
        try:
            # Controlled precondition: finish boot, then known safe state.
            dev.wait_ready(READY_WAIT)
            try:
                force_safe_state(dev)
```

```
            except (TimeoutError, AssertionError) as e:
>               pytest.fail(f"Precondition failed (environment/communication): {e}")
E               Failed: Precondition failed (environment/communication): No
response for command: RESET

conftest.py:61: Failed
=================================== FAILURES
===================================
_____________________________ test_pwm_valid_boundaries[100]
_____________________________

device = <device.Device object at 0x7fdd73fd4820>, value = 100

    @pytest.mark.parametrize("value", [0, 1, 50, 99, 100])
    def test_pwm_valid_boundaries(device, value):
        r = device.command(f"SET PWM {value}")
>       assert r["status"] == "OK"
E       AssertionError: assert 'ERROR' == 'OK'
E
E         - OK
E         + ERROR

tests/test_boundaries.py:6: AssertionError
____________________ test_status_right_after_ping_is_not_blocked
____________________

device = <device.Device object at 0x7fdd73ffd790>
record_property = <function record_property.<locals>.append_property at
0x7fdd73fe24c0>

    def test_status_right_after_ping_is_not_blocked(device, record_property):
        """evidence: PING and STATUS are sent in one write. If PING blocks the
        firmware, STATUS is answered only after PING (~1.3 s) and misses the
        1.0 s harness timeout. Every other command answers in 0.2-0.4 s."""
        replies = send_and_time(device, b"PING\nSTATUS\n", count=2)
        t_ping, _ = find_reply(replies, lambda r: r.get("command") == "PING")
        t_status, _ = find_reply(replies, lambda r: "heap" in r)
        record_property("ping_reply_s", None if t_ping is None else round(t_ping,
2))
        record_property("status_reply_s", None if t_status is None else
round(t_status, 2))
        assert t_ping is not None, f"no PING reply: {replies!r}"
        assert t_status is not None, f"no STATUS reply: {replies!r}"
>       assert t_status < HARNESS_TIMEOUT, (
            f"STATUS answered after {t_status:.2f}s (PING after {t_ping:.2f}s): PING
blocks the queue"
        )
```

```
E       AssertionError: STATUS answered after 1.46s (PING after 1.35s): PING
blocks the queue
E       assert 1.462445595999995 < 1.0

tests/test_candidates_bugs.py:81: AssertionError
_____________________ test_ping_is_not_an_outlier_vs_status
_____________________

device = <device.Device object at 0x7fdd73ffd880>
record_property = <function record_property.<locals>.append_property at
0x7fdd74047310>

    def test_ping_is_not_an_outlier_vs_status(device, record_property):
        """evidence (heuristic, not a requirement): PING should not be several
        times slower than STATUS. Medians of 5 STATUS and 3 PING requests."""
        def median_latency(cmd, n):
            values = []
            for _ in range(n):
                device.command_raw(cmd, timeout=5.0)
                values.append(device.last_elapsed)
            return statistics.median(values)

        status_s = median_latency("STATUS", 5)
        ping_s = median_latency("PING", 3)
        record_property("status_median_s", round(status_s, 2))
        record_property("ping_median_s", round(ping_s, 2))
>       assert ping_s <= 3 * status_s, (
            f"PING median {ping_s:.2f}s is {ping_s / status_s:.1f}x STATUS median
{status_s:.2f}s"
        )
E       AssertionError: PING median 1.36s is 5.7x STATUS median 0.24s
E       assert 1.3617281149999982 <= (3 * 0.2404070640000029)

tests/test_candidates_bugs.py:109: AssertionError
_____________________ test_led_on_and_off_keep_pwm_value[0]
_____________________

device = <device.Device object at 0x7fdd7404e0d0>
record_property = <function record_property.<locals>.append_property at
0x7fdd74047280>
pwm = 0

    @pytest.mark.parametrize("pwm", [0, 30, 99])
    def test_led_on_and_off_keep_pwm_value(device, record_property, pwm):
        """evidence: LED and PWM are separate outputs (requirements 2 and 7 list
        them side by side). SET LED ON/OFF must not change the PWM value.
        Measured so far: SET LED ON gives STATUS pwm 255. If this fails, report
```

```
        it as a defect candidate AND as a question to the requirement owner
        (requirement 4 only says 'output state changes accordingly')."""
        device.command(f"SET PWM {pwm}")
        before = device.command("STATUS")["pwm"]
        device.command("SET LED ON")
        after_on = device.command("STATUS")["pwm"]
        device.command("SET LED OFF")
        after_off = device.command("STATUS")["pwm"]
        record_property("pwm_before", before)
        record_property("pwm_after_led_on", after_on)
        record_property("pwm_after_led_off", after_off)
        assert before == raw_duty(pwm)
>       assert after_on == before, f"SET LED ON changed pwm {before} ->
{after_on}"
E       AssertionError: SET LED ON changed pwm 0 -> 255
E       assert 255 == 0

tests/test_candidates_bugs.py:171: AssertionError
_____________________ test_led_on_and_off_keep_pwm_value[30]
_____________________

device = <device.Device object at 0x7fdd74073af0>
record_property = <function record_property.<locals>.append_property at
0x7fdd74047a60>
pwm = 30

    @pytest.mark.parametrize("pwm", [0, 30, 99])
    def test_led_on_and_off_keep_pwm_value(device, record_property, pwm):
        """evidence: LED and PWM are separate outputs (requirements 2 and 7 list
        them side by side). SET LED ON/OFF must not change the PWM value.
        Measured so far: SET LED ON gives STATUS pwm 255. If this fails, report
        it as a defect candidate AND as a question to the requirement owner
        (requirement 4 only says 'output state changes accordingly')."""
        device.command(f"SET PWM {pwm}")
        before = device.command("STATUS")["pwm"]
        device.command("SET LED ON")
        after_on = device.command("STATUS")["pwm"]
        device.command("SET LED OFF")
        after_off = device.command("STATUS")["pwm"]
        record_property("pwm_before", before)
        record_property("pwm_after_led_on", after_on)
        record_property("pwm_after_led_off", after_off)
        assert before == raw_duty(pwm)
>       assert after_on == before, f"SET LED ON changed pwm {before} ->
{after_on}"
E       AssertionError: SET LED ON changed pwm 76 -> 255
E       assert 255 == 76
```

```
tests/test_candidates_bugs.py:171: AssertionError
_____________________ test_led_on_and_off_keep_pwm_value[99]
_____________________

device = <device.Device object at 0x7fdd73f08a90>
record_property = <function record_property.<locals>.append_property at
0x7fdd74047550>
pwm = 99

    @pytest.mark.parametrize("pwm", [0, 30, 99])
    def test_led_on_and_off_keep_pwm_value(device, record_property, pwm):
        """evidence: LED and PWM are separate outputs (requirements 2 and 7 list
        them side by side). SET LED ON/OFF must not change the PWM value.
        Measured so far: SET LED ON gives STATUS pwm 255. If this fails, report
        it as a defect candidate AND as a question to the requirement owner
        (requirement 4 only says 'output state changes accordingly')."""
        device.command(f"SET PWM {pwm}")
        before = device.command("STATUS")["pwm"]
        device.command("SET LED ON")
        after_on = device.command("STATUS")["pwm"]
        device.command("SET LED OFF")
        after_off = device.command("STATUS")["pwm"]
        record_property("pwm_before", before)
        record_property("pwm_after_led_on", after_on)
        record_property("pwm_after_led_off", after_off)
        assert before == raw_duty(pwm)
>       assert after_on == before, f"SET LED ON changed pwm {before} ->
{after_on}"
E       AssertionError: SET LED ON changed pwm 252 -> 255
E       assert 255 == 252

tests/test_candidates_bugs.py:171: AssertionError
_____________________ test_set_pwm_does_not_turn_led_on
_____________________

device = <device.Device object at 0x7fdd7405d2b0>

    def test_set_pwm_does_not_turn_led_on(device):
        """SET LED OFF, SET PWM 50, STATUS -> led must stay false.
        If it is true the LED follows PWM: ask the requirement owner (req 4)."""
        device.command("SET LED OFF")
        device.command("SET PWM 50")
        status = device.command("STATUS")
>       assert status["led"] is False
E       assert True is False
```

```
tests/test_exploratory.py:53: AssertionError
_____________________________ test_reset_clears_pwm[1]
_____________________________

device = <device.Device object at 0x7fdd73fd4760>, pwm = 1

    @pytest.mark.parametrize("pwm", [1, 50, 99])
    def test_reset_clears_pwm(device, pwm):
        device.command(f"SET PWM {pwm}")
        assert device.command("RESET")["status"] == "OK"
>       assert device.command("STATUS")["pwm"] == 0
E       assert 2 == 0

tests/test_extra.py:82: AssertionError
_____________________________ test_reset_clears_pwm[50]
_____________________________

device = <device.Device object at 0x7fdd74064af0>, pwm = 50

    @pytest.mark.parametrize("pwm", [1, 50, 99])
    def test_reset_clears_pwm(device, pwm):
        device.command(f"SET PWM {pwm}")
        assert device.command("RESET")["status"] == "OK"
>       assert device.command("STATUS")["pwm"] == 0
E       assert 127 == 0

tests/test_extra.py:82: AssertionError
_____________________________ test_reset_clears_pwm[99]
_____________________________

device = <device.Device object at 0x7fdd73fea790>, pwm = 99

    @pytest.mark.parametrize("pwm", [1, 50, 99])
    def test_reset_clears_pwm(device, pwm):
        device.command(f"SET PWM {pwm}")
        assert device.command("RESET")["status"] == "OK"
>       assert device.command("STATUS")["pwm"] == 0
E       assert 252 == 0

tests/test_extra.py:82: AssertionError
_____________________________ test_reset_turns_led_off
_____________________________

device = <device.Device object at 0x7fdd73f4cca0>

    def test_reset_turns_led_off(device):
```

```
        device.command("SET LED ON")
        assert device.command("RESET")["status"] == "OK"
>       assert device.command("STATUS")["led"] is False
E       assert True is False

tests/test_extra.py:88: AssertionError
_____________________ test_temp_is_valid_value_or_sensor_error[1]
_____________________

self = <device.Device object at 0x7fdd740a7310>, command = 'GET TEMP'
timeout = None

    def command(self, command, timeout=None):
        raw = self.command_raw(command, timeout=timeout)
        try:
>           return json.loads(raw)

device.py:88:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
/Users/Sergio/opt/anaconda3/lib/python3.9/json/__init__.py:346: in loads
    return _default_decoder.decode(s)
/Users/Sergio/opt/anaconda3/lib/python3.9/json/decoder.py:337: in decode
    obj, end = self.raw_decode(s, idx=_w(s, 0).end())
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <json.decoder.JSONDecoder object at 0x7fdd738dc2e0>
s = '{"status":"OK","temperature":24.5', idx = 0

    def raw_decode(self, s, idx=0):
        """Decode a JSON document from ``s`` (a ``str`` beginning with
        a JSON document) and return a 2-tuple of the Python
        representation and the index in ``s`` where the document ended.

        This can be used to decode a JSON document from a string that may
        have extraneous data at the end.

        """
        try:
>           obj, end = self.scan_once(s, idx)
E           json.decoder.JSONDecodeError: Expecting ',' delimiter: line 1 column 34
(char 33)

/Users/Sergio/opt/anaconda3/lib/python3.9/json/decoder.py:353:
JSONDecodeError

The above exception was the direct cause of the following exception:
```

```
device = <device.Device object at 0x7fdd740a7310>, attempt = 1

    @pytest.mark.parametrize("attempt", [1, 2, 3])
    def test_temp_is_valid_value_or_sensor_error(device, attempt):
>       r = device.command("GET TEMP")

tests/test_extra.py:101:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <device.Device object at 0x7fdd740a7310>, command = 'GET TEMP'
timeout = None

    def command(self, command, timeout=None):
        raw = self.command_raw(command, timeout=timeout)
        try:
            return json.loads(raw)
        except json.JSONDecodeError as e:
>           raise AssertionError(f"Invalid JSON from device: {raw!r}") from e
E           AssertionError: Invalid JSON from device:
'{"status":"OK","temperature":24.5'

device.py:90: AssertionError
_____ test_temp_is_valid_value_or_sensor_error[2]
_____

self = <device.Device object at 0x7fdd740c5070>, command = 'GET TEMP'
timeout = None

    def command(self, command, timeout=None):
        raw = self.command_raw(command, timeout=timeout)
        try:
>           return json.loads(raw)

device.py:88:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
/Users/Sergio/opt/anaconda3/lib/python3.9/json/__init__.py:346: in loads
    return _default_decoder.decode(s)
/Users/Sergio/opt/anaconda3/lib/python3.9/json/decoder.py:337: in decode
    obj, end = self.raw_decode(s, idx=_w(s, 0).end())
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <json.decoder.JSONDecoder object at 0x7fdd738dc2e0>
s = '{"status":"OK","temperature":24.5', idx = 0

    def raw_decode(self, s, idx=0):
        """Decode a JSON document from ``s`` (a ``str`` beginning with
```

```
        a JSON document) and return a 2-tuple of the Python
        representation and the index in ``s`` where the document ended.

        This can be used to decode a JSON document from a string that may
        have extraneous data at the end.

        """
        try:
>           obj, end = self.scan_once(s, idx)
E           json.decoder.JSONDecodeError: Expecting ',' delimiter: line 1 column 34
(char 33)

/Users/Sergio/opt/anaconda3/lib/python3.9/json/decoder.py:353:
JSONDecodeError

The above exception was the direct cause of the following exception:

device = <device.Device object at 0x7fdd740c5070>, attempt = 2

    @pytest.mark.parametrize("attempt", [1, 2, 3])
    def test_temp_is_valid_value_or_sensor_error(device, attempt):
>       r = device.command("GET TEMP")

tests/test_extra.py:101:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <device.Device object at 0x7fdd740c5070>, command = 'GET TEMP'
timeout = None

    def command(self, command, timeout=None):
        raw = self.command_raw(command, timeout=timeout)
        try:
            return json.loads(raw)
        except json.JSONDecodeError as e:
>           raise AssertionError(f"Invalid JSON from device: {raw!r}") from e
E           AssertionError: Invalid JSON from device:
'{"status":"OK","temperature":24.5'

device.py:90: AssertionError
_____________________ test_temp_is_valid_value_or_sensor_error[3]
_____________________

self = <device.Device object at 0x7fdd740c8bb0>, command = 'GET TEMP'
timeout = None

    def command(self, command, timeout=None):
        raw = self.command_raw(command, timeout=timeout)
```

```
        try:
>           return json.loads(raw)

device.py:88:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
/Users/Sergio/opt/anaconda3/lib/python3.9/json/__init__.py:346: in loads
    return _default_decoder.decode(s)
/Users/Sergio/opt/anaconda3/lib/python3.9/json/decoder.py:337: in decode
    obj, end = self.raw_decode(s, idx=_w(s, 0).end())
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <json.decoder.JSONDecoder object at 0x7fdd738dc2e0>
s = '{"status":"OK","temperature":24.5', idx = 0

    def raw_decode(self, s, idx=0):
        """Decode a JSON document from ``s`` (a ``str`` beginning with
        a JSON document) and return a 2-tuple of the Python
        representation and the index in ``s`` where the document ended.

        This can be used to decode a JSON document from a string that may
        have extraneous data at the end.

        """
        try:
>           obj, end = self.scan_once(s, idx)
E           json.decoder.JSONDecodeError: Expecting ',' delimiter: line 1 column 34
(char 33)

/Users/Sergio/opt/anaconda3/lib/python3.9/json/decoder.py:353:
JSONDecodeError

The above exception was the direct cause of the following exception:

device = <device.Device object at 0x7fdd740c8bb0>, attempt = 3

    @pytest.mark.parametrize("attempt", [1, 2, 3])
    def test_temp_is_valid_value_or_sensor_error(device, attempt):
>       r = device.command("GET TEMP")

tests/test_extra.py:101:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <device.Device object at 0x7fdd740c8bb0>, command = 'GET TEMP'
timeout = None

    def command(self, command, timeout=None):
        raw = self.command_raw(command, timeout=timeout)
```

```
        try:
            return json.loads(raw)
        except json.JSONDecodeError as e:
>           raise AssertionError(f"Invalid JSON from device: {raw!r}") from e
E           AssertionError: Invalid JSON from device:
'{"status":"OK","temperature":24.5'

device.py:90: AssertionError
_____________________________ test_reset_restores_safe_state
_____________________________

device = <device.Device object at 0x7fdd740bd850>

    def test_reset_restores_safe_state(device):
        device.command("SET LED ON")
        device.command("RESET")
        status = device.command("STATUS")
>       assert status["pwm"] == 0
E       assert 255 == 0

tests/test_relay.py:9: AssertionError
_____________________________ test_temperature_is_valid_json
_____________________________

self = <device.Device object at 0x7fdd740c0550>, command = 'GET TEMP'
timeout = None

    def command(self, command, timeout=None):
        raw = self.command_raw(command, timeout=timeout)
        try:
>           return json.loads(raw)

device.py:88:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
/Users/Sergio/opt/anaconda3/lib/python3.9/json/__init__.py:346: in loads
    return _default_decoder.decode(s)
/Users/Sergio/opt/anaconda3/lib/python3.9/json/decoder.py:337: in decode
    obj, end = self.raw_decode(s, idx=_w(s, 0).end())
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <json.decoder.JSONDecoder object at 0x7fdd738dc2e0>
s = '{"status":"OK","temperature":24.5', idx = 0

    def raw_decode(self, s, idx=0):
        """Decode a JSON document from ``s`` (a ``str`` beginning with
        a JSON document) and return a 2-tuple of the Python
        representation and the index in ``s`` where the document ended.
```

```
        This can be used to decode a JSON document from a string that may
        have extraneous data at the end.

        """
        try:
>           obj, end = self.scan_once(s, idx)
E           json.decoder.JSONDecodeError: Expecting ',' delimiter: line 1 column 34
(char 33)

/Users/Sergio/opt/anaconda3/lib/python3.9/json/decoder.py:353:
JSONDecodeError

The above exception was the direct cause of the following exception:

device = <device.Device object at 0x7fdd740c0550>

    def test_temperature_is_valid_json(device):
>       r = device.command("GET TEMP")

tests/test_sensor.py:2:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <device.Device object at 0x7fdd740c0550>, command = 'GET TEMP'
timeout = None

    def command(self, command, timeout=None):
        raw = self.command_raw(command, timeout=timeout)
        try:
            return json.loads(raw)
        except json.JSONDecodeError as e:
>           raise AssertionError(f"Invalid JSON from device: {raw!r}") from e
E           AssertionError: Invalid JSON from device:
'{"status":"OK","temperature":24.5'

device.py:90: AssertionError
_____________________________________ test_ping
_____________________________________

device = <device.Device object at 0x7fdd740768e0>

    def test_ping(device):
>       r = device.command("PING")

tests/test_status.py:2:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
device.py:86: in command
```

```
    raw = self.command_raw(command, timeout=timeout)
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <device.Device object at 0x7fdd740768e0>, command = 'PING'
timeout = None

    def command_raw(self, command, timeout=None):
        """Send one command and return the first reply line.
        `timeout` overrides the read timeout for this call only."""
        old_timeout = self.ser.timeout
        if timeout is not None:
            self.ser.timeout = timeout
        try:
            self.ser.reset_input_buffer()
            self.log("TX", command)
            start = time.monotonic()
            self.ser.write((command + "\n").encode())
            self.ser.flush()
            line = self.ser.readline().decode(errors="replace").strip()
            elapsed = time.monotonic() - start
        finally:
            self.ser.timeout = old_timeout
        if not line:
            self.timed_out = True
            self.log("RX", f"<no response, timeout after {elapsed:.2f}s>")
>           raise TimeoutError(f"No response for command: {command}")
E           TimeoutError: No response for command: PING

device.py:80: TimeoutError
=========================== short test summary info ============================
SKIPPED [1] tests/test_exploratory.py:128: set RUN_SLOW=1 to run slow tests
SKIPPED [1] tests/test_exploratory.py:140: disconnect the temperature sensor
in diagram.json, restart Wokwi, set SENSOR_DISCONNECTED=1
XFAIL tests/test_extra.py::test_ping_latency_is_below_harness_timeout[1] -
OBS-1: PING answers in ~1.3 s, harness timeout is 1.0 s (no latency
requirement)
XFAIL tests/test_extra.py::test_ping_latency_is_below_harness_timeout[2] -
OBS-1: PING answers in ~1.3 s, harness timeout is 1.0 s (no latency
requirement)
XFAIL tests/test_extra.py::test_ping_latency_is_below_harness_timeout[3] -
OBS-1: PING answers in ~1.3 s, harness timeout is 1.0 s (no latency
requirement)
XFAIL tests/test_extra.py::test_command_answers_within_timeout[PING] -
OBS-1: PING answers in ~1.3 s, harness timeout is 1.0 s (no latency
requirement)
ERROR tests/test_timeout.py::test_no_response_becomes_timeout - Failed:
```

```
Preco...
FAILED tests/test_boundaries.py::test_pwm_valid_boundaries[100] -
AssertionEr...
FAILED tests/
test_candidates_bugs.py::test_status_right_after_ping_is_not_blocked
FAILED tests/test_candidates_bugs.py::test_ping_is_not_an_outlier_vs_status
FAILED tests/
test_candidates_bugs.py::test_led_on_and_off_keep_pwm_value[0]
FAILED tests/
test_candidates_bugs.py::test_led_on_and_off_keep_pwm_value[30]
FAILED tests/
test_candidates_bugs.py::test_led_on_and_off_keep_pwm_value[99]
FAILED tests/test_exploratory.py::test_set_pwm_does_not_turn_led_on -
assert ...
FAILED tests/test_extra.py::test_reset_clears_pwm[1] - assert 2 == 0
FAILED tests/test_extra.py::test_reset_clears_pwm[50] - assert 127 == 0
FAILED tests/test_extra.py::test_reset_clears_pwm[99] - assert 252 == 0
FAILED tests/test_extra.py::test_reset_turns_led_off - assert True is False
FAILED tests/test_extra.py::test_temp_is_valid_value_or_sensor_error[1] -
Ass...
FAILED tests/test_extra.py::test_temp_is_valid_value_or_sensor_error[2] -
Ass...
FAILED tests/test_extra.py::test_temp_is_valid_value_or_sensor_error[3] -
Ass...
FAILED tests/test_relay.py::test_reset_restores_safe_state - assert 255 == 0
FAILED tests/test_sensor.py::test_temperature_is_valid_json -
AssertionError:...
FAILED tests/test_status.py::test_ping - TimeoutError: No response for
comman...
=== 17 failed, 72 passed, 2 skipped, 4 xfailed, 1 error in 623.14s (0:10:23) ===
(.venv) (base) Sergio@SERHIIs-MacBook-Pro automation % python
manual_check.py rfc2217://localhost:4000
READY banner seen: False

--- PING x5: how late is the answer?
PING       -> {"status":"OK","command":"PING"}  [1.40s]
PING       -> {"status":"OK","command":"PING"}  [1.36s]
PING       -> {"status":"OK","command":"PING"}  [1.36s]
PING       -> {"status":"OK","command":"PING"}  [1.35s]
PING       -> {"status":"OK","command":"PING"}  [1.35s]

--- GET TEMP x3: is the closing brace always missing?
GET TEMP    -> {"status":"OK","temperature":24.5  [0.32s]
GET TEMP    -> {"status":"OK","temperature":24.5  [0.23s]
GET TEMP    -> {"status":"OK","temperature":24.5  [0.23s]

--- PWM boundary
```

```
SET PWM 99   -> {"status":"OK","pwm":99}  [0.29s]
SET PWM 100  -> {"status":"ERROR","code":"INVALID_VALUE"}  [0.35s]
SET PWM 101  -> {"status":"ERROR","code":"INVALID_VALUE"}  [0.33s]
STATUS       -> {"status":"OK","led":true,"pwm":252,"heap":349852}  [0.34s]

--- PWM scale in STATUS
SET PWM 1    -> {"status":"OK","pwm":1}  [0.44s]
STATUS       -> {"status":"OK","led":true,"pwm":2,"heap":349852}  [0.34s]
SET PWM 50   -> {"status":"OK","pwm":50}  [0.34s]
STATUS       -> {"status":"OK","led":true,"pwm":127,"heap":349852}  [0.35s]
SET PWM 99   -> {"status":"OK","pwm":99}  [0.37s]
STATUS       -> {"status":"OK","led":true,"pwm":252,"heap":349852}  [0.35s]

--- RESET after LED ON and PWM 50
SET LED ON   -> {"status":"OK","led":true}  [0.27s]
SET PWM 50   -> {"status":"OK","pwm":50}  [0.27s]
STATUS       -> {"status":"OK","led":true,"pwm":127,"heap":349852}  [0.34s]
RESET        -> {"status":"OK","reset":true}  [0.46s]
STATUS       -> {"status":"OK","led":true,"pwm":127,"heap":349852}  [0.32s]

--- Unknown command
FOO          -> {"status":"ERROR","code":"UNKNOWN_COMMAND"}  [0.42s]

--- NO_RESPONSE (must time out)
NO_RESPONSE  -> NO ANSWER within 5 s

Saved evidence/manual_check.log
(.venv) (base) Sergio@SERHIIs-MacBook-Pro automation %
```